

AI, Agents & AI-Coding-Tools: Ein Deep Dive für die Praxis

Wie AI-Agents, Multi-Agent-Orchestrierung und moderne Coding-Tools die Softwareentwicklung verändern. Ein praxisnaher Einblick für Engineering- und IT-Teams.

REFERENT

Sebastian Kehle

ROLLE

AI Engineer & Gründer, webvise

DAUER

60 Minuten

FORMAT

Vortrag + Q&A

Sebastian Kehle

Gründer von webvise und AI Engineer bei luca in Berlin. Mein Hintergrund liegt in Design und Kommunikation - über die Jahre bin ich immer tiefer in Softwareentwicklung und Automatisierung eingestiegen.

Bei luca arbeite ich an LLM-basierten Lösungen für die Hospitality-Branche. Bei webvise verbinde ich diese Erfahrung mit Kundenprojekten: Webentwicklung, Design und zunehmend AI-Automatisierung.

Was ich heute vorstelle, sind Tools und Workflows, die ich im Alltag einsetze.

TypeScriptReactNext.jsNode.jsPostgreSQLOpenAIAnthropic

Vercel AI SDKMastran8n

2025 - heute	AI Engineer, luca LLM-Integrationen, Tooling, Automatisierung
2022 - 2025	Frontend Engineer, luca B2B-Features, Design-Kollaboration
2016 - heute	Gründer, webvise Design, Entwicklung, AI-Automatisierung
2017 - 2021	CEO, Addisco Education Recruiting-Plattform, Matching-Algorithmus

Warum gerade jetzt?

Der Wendepunkt

Wir haben den Punkt überschritten, an dem AI ein Experiment war. AI ist Infrastruktur geworden. Jede grosse IDE hat AI-Features. Terminal-Agents arbeiten autonom. Die Frage hat sich verschoben: nicht mehr "Sollen wir AI nutzen?" sondern "Wie schnell können wir adoptieren?"

Die Tools sind reif. Die Modelle sind gut genug. Die Integration in bestehende Workflows funktioniert. Was fehlt, ist eine strukturierte Einführung - und genau darum geht es heute.

20-45%

Produktivitätssteigerung
McKinsey, 2025

55%

Schnellere
Task-
Completion
GitHub, 2025

76%

der Entwickler
nutzen
AI-Tools
*Stack Overflow,
2025*

75%

Enterprise-
Engineers
bis 2028 mit AI
Gartner, 2026

Drei Wellen der AI-gestützten Entwicklung

Jede Welle hat die Rolle des Entwicklers fundamental verändert. Wir sind mitten in Welle 3.

Welle 01 Autocomplete 2021-2023. GitHub Copilot bringt AI in den Editor. Zeilenvervollständigung, Boilerplate. Der Entwickler bleibt am Steuer, AI ist ein besseres IntelliSense.	Welle 02 Chat & Copilot 2023-2025. ChatGPT, Claude, Cursor. Konversationen über Code, Multi-File-Kontext. Ganze Funktionen beschreiben und generieren lassen.	Welle 03 Autonome Agents 2025-heute. AI-Systeme, die planen, Tools nutzen, testen, deployen. Nicht ein Prompt, eine Antwort - sondern ein Ziel, viele Schritte. Das ist jetzt.
---	---	--

Der entscheidende Unterschied: In Welle 1 und 2 reagiert AI auf den Entwickler. In Welle 3 agiert AI eigenständig - plant, iteriert, korrigiert sich selbst.

AI Coding Tools 2026

Co

GitHub Copilot

Breiteste Adoption, solide Autocomplete-Qualität. Agent Mode seit 2025, wirkt aber nachträglich angefügt. Für einfache Vervollständigung gut - bei komplexen Aufgaben schnell am Limit.

Cu

Cursor

VS-Code-Fork mit nativer AI. Starkes Multi-File-Editing, guter Codebase-Kontext, Composer für komplexe Aufgaben. Aktuell eine der besten IDE-Erfahrungen.

Cl

Claude Code

Terminal-basierter autonomer Agent. Plant, implementiert, testet eigenständig. Exzellentes Codebase-Verständnis. Volle Git/Shell/API-Integration. Selbst-hostbar.

Wi

Windsurf

AI-First-IDE mit Flows für Multi-Step-Aufgaben. Niedrige Einstiegshürde, gutes Kontext-Management. Solide Alternative zu Cursor.

Cx

Codex CLI

OpenAIs Terminal-Agent. Cloud-Sandbox-Execution, OpenAI-Ökosystem. Noch relativ jung im Vergleich zu Claude Code.

Ge

Gemini CLI

Googles Terminal-Agent. Grösstes Kontextfenster (1M+ Tokens), Google-Ökosystem. Kostenloses Tier verfügbar.

Wie sie sich unterscheiden

Kein Tool ist in allem das beste. Die Wahl hängt vom Use Case ab.

	COPILOT	CURSOR	CLAUDE CODE	WINDSURF
AUTONOMIE				
CODEBASE-VERSTÄNDNIS				
KOMPLEXE AUFGABEN				
ENTERPRISE-FEATURES				
COMPLIANCE				

Meine Empfehlung: Cursor oder Windsurf als IDE-Einstieg. Claude Code für komplexe, autonome Aufgaben. Copilot nur, wenn es bereits im Unternehmen etabliert ist - ein Wechsel lohnt sich.

Was ist ein AI Agent?

Ein Chatbot antwortet auf Fragen.
Ein Agent erledigt Aufgaben.

1

Zielverständnis

Versteht komplexe Ziele, zerlegt sie selbstständig in Teilschritte.

2

Tool-Nutzung

Greift auf Dateisystem, APIs, Datenbanken, Browser, Terminal zu.

3

Iteration

Prüft Ergebnisse, korrigiert Fehler, versucht Alternativen.

Der Agent-Loop

1. Ziel verstehen
 2. Plan erstellen
 3. Tools auswählen
 4. Ausführen
 5. Ergebnis prüfen
 6. Korrigieren oder abschliessen
- ↳ Wiederholen bis Ziel erreicht

Anders als ein Chatbot gibt sich ein Agent nicht mit der ersten Antwort zufrieden. Er iteriert, bis das Ziel nachweisbar erreicht ist.

Model Context Protocol - USB-C für AI

MCP ist ein offener Standard, der definiert, wie AI-Modelle mit externen Tools kommunizieren. Ein Protokoll, alle Tools.

Vor MCP

- Jedes Tool eigene Integration
- Keine Standardisierung
- Kein Austausch zwischen Tools

Mit MCP

- Ein Protokoll für alle
- Einmal bauen, überall nutzen
- Kein Vendor-Lock-in

Für Unternehmen: MCP-Server für interne Systeme werden einmal gebaut und von allen AI-Tools genutzt. Keine doppelten Integrationen.

MCP-Server in der Praxis

Datenbank-Server

PostgreSQL, Redis, Elasticsearch - sicherer AI-Zugriff

Git-Server

Repository-Historie, Branches, Pull Requests

Jira/Linear-Server

Tickets lesen, erstellen, Status aktualisieren

Monitoring-Server

Grafana-Dashboards, Logs, Alerts analysieren

Browser-Server

Webseiten navigieren, UIs testen, Daten extrahieren

Claude Code + oh-my-claudecode

Claude Code

Läuft im Terminal. Zugriff auf das gesamte Projekt-Dateisystem, Git, Shell-Kommandos und MCP-Server. Mein primäres Entwicklungstool.

- Versteht die gesamte Codebase
- Erstellt Commits, Branches, Pull Requests
- Führt Tests und Builds aus
- Greift auf Datenbanken und APIs zu (via MCP)

oh-my-claudecode (OMC)

Eine Orchestrierungsschicht auf Claude Code. Koordiniert spezialisierte Agents für verschiedene Aufgaben - automatisch.

- Multi-Agent-Koordination
- Parallele Ausführung
- Spezialisierte Rollen (Architect, Reviewer...)
- Skills-Integration

Das ist kein Demo-Setup. So liefere ich Code - bei luca und bei webvise-Kundenprojekten.

Agent-Katalog

Jeder Agent hat eine klare Rolle. Wie ein Team aus Spezialisten, die parallel arbeiten.

 Architect Strategische Architektur-Entscheidungen. Read-Only. Prüft Pläne bevor Code geschrieben wird.	 Executor Fokussierte Implementierung. Arbeitet parallel an unabhängigen Aufgaben. Schnell und präzise.	 Code Reviewer Detaillierte Reviews mit Severity-Rating. Findet Patterns, die Menschen übersehen.
 Test Engineer Test-Strategie und TDD-Workflows. Schreibt und validiert Tests automatisch.	 Security Reviewer OWASP Top 10, Secrets-Detection, unsichere Patterns. Automatisierte Security-Checks.	 Verifier Evidenz-basierte Completion-Checks. Tests bestanden, keine Regressionen, Deployment OK.

Von der Anforderung zum Deployment

```
// Anforderung: "Implementiere User-Authentifizierung für die API"
```

1. **Planner** → Analysiert bestehende Architektur, identifiziert Auth-Patterns
2. **Architect** → Reviewt Plan, empfiehlt JWT + Session-Strategie
3. **Executors** → Arbeiten parallel:
 - └ Agent 1: Auth-Middleware + Route Guards
 - └ Agent 2: User-Model + Migration
 - └ Agent 3: Test-Suite + Dokumentation
4. **Reviewer** → Prüft Code-Qualität, Security-Implikationen
5. **Verifier** → Alle Tests grün, keine Regressionen, merge-ready

Zeitersparnis: ~45 Minuten für eine Aufgabe, die typischerweise 1-2 Tage dauert.

Qualität: Jeder Schritt wird von einem spezialisierten Agent geprüft. Menschliches Review bleibt Pflicht.

Wiederverwendbare Agent-Fähigkeiten

Skills sind Markdown-basierte Anweisungen, die Agents spezifische Fähigkeiten verleihen.

- Portabel - Claude Code, Cursor, Copilot, 19+ Tools
- Versionierbar - in Git, teilbar über Teams
- Komponierbar - Skills rufen andere Skills auf

skills.sh Ökosystem

Offener Marktplatz für AI-Agent-Skills. Statt dass jedes Team eigene Prompts schreibt, gibt es eine wachsende Bibliothek wiederverwendbarer Fähigkeiten.

Für Unternehmen: Einmal einen Standard-Skill erstellen, und jeder Entwickler nutzt ihn - unabhängig von IDE oder AI-Tool.

Praxis-Beispiele

webvise-conventions Tech-Stack, Design-System, Code-Patterns. Jeder Agent kennt unsere Standards.	webvise-qa 7-Kategorien QA: Performance, Accessibility, SEO, Responsive, Analytics.
webvise-deliver 6-Phasen Workflow: Discover, Plan, Build, SEO, QA, Deliver.	Enterprise-Skills Coding-Standards, Checklisten, Onboarding. Versioniert, für alle Teams.

Case Study: AI-gestützte Auslieferung

Die Aufgabe

Komplette Webplattform mit AI-Chatbot, Mehrsprachigkeit (8 Sprachen) und automatisierten Content-Workflows für ein Bauunternehmen.

Klassische Schätzung: 6-8 Wochen.

Das Ergebnis

- Next.js-Plattform mit dynamischer Projektgalerie
- AI-Chatbot (Google Gemini) für Besucheranfragen
- 8-Sprachen-Support für internationales Team
- Automatisierte Content-Pipelines

Kennzahlen

2 Wochen Delivery	95+ PageSpeed Score	8 Sprachen ab Launch	AI Chatbot integriert
-------------------------	---------------------------	----------------------------	-----------------------------

Next.js	Gemini	i18n	Vercel	TailwindCSS
---------	--------	------	--------	-------------

Schlüssel: AI-gestützte Entwicklung mit webvise-Skills für QA und Delivery. Test-Coverage ab Tag 1.

Was AI kann - und was nicht

Glaubwürdigkeit entsteht durch Ehrlichkeit. AI ist kein Zauberstab.

AI ist stark bei

- Code-Generierung und Boilerplate
- Test-Erstellung und Coverage
- Dokumentation generieren
- Refactoring und Code-Migration
- Pattern-Erkennung und Security-Scans
- Dependency-Management
- Code-Review-Unterstützung

AI braucht Menschen für

- Architektur-Entscheidungen
- Business-Logik und Domänenwissen
- User Research und Produktstrategie
- Edge-Case-Beurteilung
- Team-Dynamik und Kommunikation
- Ethische Abwägungen
- Kreative Problemlösung auf hohem Niveau

Die besten Ergebnisse entstehen, wenn man AI wie einen sehr fähigen Junior-Entwickler behandelt - schnell, gründlich, aber braucht Richtung und Review.

EU AI Act: Was Sie wissen müssen

In Kraft seit August 2024, volle Anwendung ab August 2026.

Minimales Risiko

Die meisten AI-Coding-Tools: Autocomplete, Refactoring, Dokumentation. Keine besonderen Anforderungen, aber Transparenzpflichten.

Begrenztes Risiko

Agent-Systeme, die eigenständig deployen oder Infrastruktur-Entscheidungen treffen. Transparenz- und Dokumentationspflicht.

Hohes Risiko

AI in sicherheitskritischen Anwendungen. Wenn AI-Code in Prüf- oder Zertifizierungssoftware fließt. Menschliche Aufsicht erforderlich.

Empfehlung: Applikationen nach Risikoklassen klassifizieren. Richtlinien pro Risikoklasse definieren. AI-Governance-Board etablieren, das Standards setzt und überwacht.

DSGVO, Datenflüsse & Anbieter

Wo gehen die Daten hin?

Wenn Entwickler AI-Tools nutzen, wird Quellcode an den Anbieter gesendet. Das ist relevant, wenn:

- Code personenbezogene Daten enthält
- Proprietäre Geschäftslogik betroffen ist
- Regulatorische Vorgaben gelten

Empfehlungen

- API-Zugang nutzen (kein Training auf Ihren Daten)
- Data Processing Agreements abschliessen
- Code-Scanning vor dem Senden (keine Secrets, keine PII)
- EU-Hosting-Optionen prüfen

Anbieter-Übersicht

ANBIETER	TRAINING	DPA	EU
Anthropic	Nein	Ja	Geplant
OpenAI	Nein	Ja	Azure EU
GitHub	Nein	Ja	Nein
Google	Nein	Ja	Ja

Die Air-Gap-Option

Warum lokal?

Vollständig on-premise. Kein Byte verlässt das Netzwerk. Ideal für:

- Sicherheitskritischer Code
- IP-sensitive Projekte
- Regulierte Branchen

Deployment-Tools

- Ollama - Einfachstes Setup, Mac/Linux/Windows
- llama.cpp - Maximale Performance, C++ basiert
- vLLM - Production-grade Serving für Teams

Hybrid-Ansatz: Lokale Modelle für sensiblen Code. Cloud-APIs für unkritische Entwicklung. Klare Richtlinien.

Verfügbare Modelle

Qwen 2.5 Coder

Alibaba | 7B-32B | 16-64GB RAM

Sehr gut

DeepSeek Coder V3

DeepSeek | 16B-236B | 32-128GB+ RAM

Exzellent

Mistral Codestral

Mistral | 22B | 32-64GB RAM

Sehr gut

StarCoder2

BigCode | 3B-15B | 8-32GB RAM

Gut

Audit-Trails & Governance

Nachvollziehbarkeit ist nicht optional. AI-generierter Code muss denselben Standards entsprechen.

G Git-Integration Alle AI-Änderungen gehen durch normale Git-Workflows. Branches, PRs, Reviews. Nichts umgeht den Prozess.	T Co-Author-Tags AI-generierte Commits werden markiert (Co-Authored-By: Claude). Volle Transparenz, welcher Code von welchem Tool stammt.	R Review-Pflicht Kein AI-Code geht ohne menschliches Review in Produktion. Derselbe Standard wie für jeden Entwickler.
L Logging Welches Modell, welcher Prompt, welcher Output. Für kritische Systeme vollständig dokumentiert.	Q Test-Standards AI-Code erfüllt dieselben (oder höhere) Test-Anforderungen. Coverage-Ziele gelten unverändert.	P Tooling Git-Hooks für Co-Author-Tags, CI/CD-Pipelines die AI-Code flaggen, Custom MCP-Server für Logging.

Crawl - Walk - Run

Strukturierte Einführung. Schritt für Schritt, messbar, kontrolliert.

PHASE 01 / CRAWL Erfahrung sammeln Monate 1-3. 5-10 Entwickler. • Cursor oder Windsurf (niedrige Hürde) • Code-Completion, Docs, Unit-Tests • Basis-Richtlinien definieren • Entwickler-Zufriedenheit messen	PHASE 02 / WALK Standards etablieren Monate 4-9. 50-100 Entwickler. • Claude Code für komplexe Aufgaben • Erste MCP-Server und Skills • AI-Richtlinie, DPA mit Anbietern • Proxy-Layer für Logging	PHASE 03 / RUN AI als Standard Ab Monat 10. Alle Teams. • Multi-Agent-Workflows • Automatisierte QA-Pipelines • Governance-Framework, Audits • ROI pro Team messen
---	---	---

Quick Wins

Sofort umsetzbar

Legacy-Dokumentation

AI generiert Docs für undokumentierten Code. Sofortiger Wissenstransfer.

Test-Coverage erhöhen

AI-generierte Tests für bestehenden Code. Coverage-Lücken schliessen.

Code-Reviews beschleunigen

AI als erster Reviewer. Menschliches Review bleibt, wird aber fokussierter.

Dependency-Updates

Automatisierte Updates und Security-Patches. Weniger manuelle Wartung.

Mittelfristig

MCP-Server für interne Systeme

Jira, CI/CD, Monitoring - AI-Zugriff auf bestehende Tools.

Shared Skills-Repository

Unternehmensweite Standards als wiederverwendbare Skills.

AI-gestützte Incident-Response

Agents analysieren Logs, identifizieren Root Causes.

Legacy-Modernisierung

AI versteht, dokumentiert und modernisiert Legacy-Code.

Was kommt als Nächstes?

Kurzfristig (2026)

- AI-Agents Standard in jeder IDE
- MCP als De-facto-Standard
- Lokale Modelle erreichen Cloud-Qualität
- Enterprise-Features bei allen Anbietern

Mittelfristig (2027)

- Multi-Agent-Teams als Normalität
- AI-gestützte Legacy-Migration im grossen Stil
- Spezialisierte Branchenmodelle
- Compliance-Checks direkt in AI-Workflows

Die Frage ist nicht ob,
sondern wie schnell.

5 Empfehlungen

1. Jetzt starten. Die Lernkurve ist flacher als gedacht.
2. Klein starten. Ein Pilot-Team, ein Tool, klare Metriken.
3. In Governance investieren. Jetzt einfacher als später.
4. Know-how aufbauen. Wissen, nicht nur Lizenzen.
5. Ergebnisse messen. KPIs ab Tag 1.

Lassen Sie uns sprechen.

Ich unterstütze Unternehmen dabei, AI sinnvoll in Entwicklungsprozesse zu integrieren. Von der Strategie bis zur Implementierung.

Sebastian Kehle

sebastian.kehle@webvise.io

webvise.io / sebastiankehle.com

LinkedIn: [/in/sebastiankehle](https://in/sebastiankehle)

GitHub: [/sebastiankehle](https://sebastiankehle)

Weiterführende Artikel

- [Best Local AI Models for Compliant Businesses](#)
- [How AI Agents Are Transforming Business Automation](#)
- [What Is the Model Context Protocol \(MCP\)](#)
- [AI Regulations and Certifications in Germany](#)
- [AI Agent Skills Directory](#)
- [How We Use AI to Build Better Software, Faster](#)

Offizielle Docs

- [Claude Code - claude.ai/claude-code](https://claude.ai/claude-code)
- [Model Context Protocol - modelcontextprotocol.io](https://modelcontextprotocol.io)
- skills.sh - Offener Skills-Marktplatz
- [EU AI Act - artificialintelligenceact.eu](https://artificialintelligenceact.eu)